

LAPORAN PROJEK
IMPLEMENTASI PEMROSESAN PARALEL MENGGUNAKAN
MPI UNTUK OPTIMASI ALGORITMA APRIORI PADA DATA
TRANSAKSI PRODUK KEBUTUHAN HARIAN

Dosen Pengampu: Heri Wijayanto, S.T.,M.T., Ph.D.



Disusun Oleh:

I Made Priyandika Adiyatma Putra Sari	F1D02410008
Lalu Abdullah Rafi Asyari	F1D02410012
Muhamad Farhan Maulana	F1D02410015
Revano Januar Adiguna Prawira	F1D02410146

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM

2026

BAB I

PENDAHULUAN

1. Latar Belakang

Perkembangan teknologi informasi pada era digital saat ini membawa dampak besar terhadap berbagai bidang, terutama pada sektor perdagangan dan bisnis. Setiap aktivitas penjualan yang dilakukan di toko modern, minimarket, supermarket, maupun platform digital akan menghasilkan data transaksi yang terus bertambah setiap hari. Data transaksi tersebut berisi informasi mengenai jenis produk yang dibeli, jumlah pembelian, waktu transaksi, serta kombinasi produk yang sering dibeli secara bersamaan oleh konsumen. Pada produk kebutuhan harian seperti beras, gula, minyak goreng, sabun, deterjen, telur, susu, dan kebutuhan rumah tangga lainnya, jumlah transaksi yang terjadi cenderung sangat tinggi karena produk tersebut selalu dibutuhkan masyarakat. Jika dikelola dengan baik, data transaksi tersebut dapat menjadi sumber informasi penting bagi perusahaan dalam mengambil keputusan bisnis [1].

Salah satu cara untuk memanfaatkan data transaksi adalah melalui teknik data mining. Data mining merupakan proses menggali informasi tersembunyi dari kumpulan data besar untuk menemukan pola tertentu yang berguna. Salah satu metode yang banyak digunakan dalam data mining adalah algoritma Apriori. Algoritma Apriori memiliki kelemahan utama, yaitu waktu pemrosesan yang cukup lama ketika digunakan pada data berukuran besar. Hal ini disebabkan oleh algoritma Apriori yang harus melakukan proses pencarian kombinasi itemset secara berulang dan menghitung nilai support dari setiap kombinasi. Semakin banyak jumlah transaksi dan jenis produk yang dianalisis, semakin besar pula beban komputasi yang diperlukan. Akibatnya, proses analisis menjadi lambat dan kurang efisien jika hanya dijalankan secara serial menggunakan satu prosesor. Kondisi ini menjadi tantangan tersendiri bagi perusahaan atau peneliti yang membutuhkan hasil analisis secara cepat dan akurat [2].

Untuk mengatasi permasalahan tersebut, salah satu solusi yang dapat diterapkan adalah pemrosesan paralel menggunakan Message Passing Interface (MPI). MPI merupakan standar komunikasi yang digunakan dalam sistem komputasi paralel, di mana suatu pekerjaan dapat dibagi ke dalam beberapa proses yang berjalan secara bersamaan. Dengan menggunakan MPI, data transaksi dapat dibagi ke dalam beberapa bagian, kemudian setiap proses menghitung support itemset secara paralel. Setelah proses selesai, hasil perhitungan dari masing-masing proses akan digabungkan menjadi satu hasil akhir. Pendekatan ini mampu mengurangi waktu eksekusi program secara signifikan dibandingkan dengan pemrosesan serial [3].

2. Tujuan

Adapun tujuan dari pengembangan program ini adalah sebagai berikut:

1. Mengimplementasikan algoritma Apriori berbasis MPI untuk pencarian frequent itemsets pada data transaksi produk kebutuhan harian secara paralel.
2. Meningkatkan efisiensi waktu eksekusi dibandingkan dengan pendekatan sekuensial.
3. Menguji performa algoritma terhadap variasi jumlah proses dan nilai *minimum support*.
4. Menyediakan sarana analitik untuk mendukung pengambilan keputusan bisnis, seperti penataan produk dan strategi *bundling*.

BAB II

DASAR TEORI

1. Algoritma Apriori

Algoritma Apriori diperkenalkan oleh Agrawal dan Srikant (1994) untuk menemukan himpunan item yang sering muncul (*frequent itemsets*) dalam sebuah set data. Algoritma Apriori adalah algoritma klasik dalam data mining untuk menemukan *frequent itemsets* berdasarkan nilai minimum *support* dan *confidence*. Algoritma ini bekerja secara iteratif atau disebut dengan *level-wise search*, di mana k *itemset* digunakan untuk menemukan $(k + 1)$ *itemset* [5].

2. Komputasi parallel dengan MPI

Message Passing Interface (MPI) adalah standar komunikasi dalam komputasi paralel. Dalam konteks apriori, MPI membagi dataset menjadi potongan-potongan (*chunk*) yang disebar ke setiap proses. Masing-masing proses menghitung frekuensi *itemset* secara lokal, kemudian hasilnya dikumpulkan (*gather*) untuk diolah secara global oleh proses utama (*root*) [3].

Dengan menggunakan MPI, data transaksi dapat dibagi ke dalam beberapa bagian, kemudian setiap proses menghitung support itemset secara paralel. Setelah proses selesai, hasil perhitungan dari masing-masing proses akan digabungkan menjadi satu hasil akhir. Pendekatan ini mampu mengurangi waktu eksekusi program secara signifikan dibandingkan dengan pemrosesan serial [4].

3. Penerapan pada Dataset Skala Besar

Penerapan algoritma Apriori pada dataset skala besar memiliki tantangan utama berupa *bottleneck* pada penggunaan memori dan intensitas pembacaan data yang berulang-ulang. Ketika jumlah transaksi mencapai jutaan, pemrosesan sekuensial tunggal sering kali mengalami kegagalan sistem atau waktu eksekusi yang tidak masuk akal. Pada dataset skala besar, data transaksi dibagi menjadi beberapa partisi kecil yang dapat dimuat ke dalam memori lokal masing-masing *node* dalam kluster paralel. Setiap *node* kemudian menghitung *local support* untuk setiap *itemset*. Strategi ini mengurangi beban memori pada satu unit pemroses pusat dan memungkinkan pemindaian data dilakukan secara simultan di seluruh infrastruktur jaringan [4].

4. Data mining

Data mining adalah suatu istilah yang digunakan untuk menguraikan penemuan pengetahuan di dalam basis data. *Data mining* adalah proses yang menggunakan teknik statistik, matematika, kecerdasan buatan, dan *machine learning* untuk mengekstraksi dan mengidentifikasi informasi yang bermanfaat dan pengetahuan yang terkait dari berbagai basis data besar [1]. *Data mining* dibagi menjadi beberapa kelompok berdasarkan tugas/pekerjaan yang dapat dilakukan, tapi pada tugas ini kami menerapkan asosiasi. Asosiasi dalam *data mining* adalah untuk menemukan atribut yang muncul dalam satu waktu. Salah satu implementasi dari asosiasi adalah *market basket analysis* atau analisis keranjang belanja [5].

BAB III

METODE

Algoritma ini menggunakan proses inisialisasi MPI untuk menemukan frekuensi itemset, di mana kombinasi item yang sering muncul bersama dalam sekumpulan transaksi menjadi dasar pembentukan association rules dalam analisis keranjang belanja (market basket analysis).

Selanjutnya, proses yang dilakukan untuk menentukan bagian mana dari pekerjaan yang harus dilakukan oleh prosesor tersebut. Prosesor dengan rank 0 adalah yang paling utama yang bertanggung jawab membaca data, membentuk kandidat, mendistribusikan pekerjaan, dan mencetak hasil akhir. Berdasarkan total transaksi yang valid, proses ini kemudian menghitung nilai minimum support absolut (jumlah) yang diperoleh dari persentase minimum support dikalikan jumlah transaksi.

Dataset yang telah diseleksi tersebut kemudian dibagi sehingga menjadi beberapa bagian sesuai jumlah proses menggunakan operasi scatter. Setiap prosesor *worker* menerima potongan data transaksi tersebut dan melakukan perhitungan frekuensi kemunculan (*local support count*) secara mandiri dan simultan. Setelah perhitungan lokal selesai, setiap prosesor mengirimkan kembali hasilnya ke prosesor utama (Rank 0) untuk diakumulasikan menjadi nilai frekuensi global. Nilai global ini dievaluasi berdasarkan ambang batas kemunculan absolut untuk menyaring kombinasi barang yang lolos menjadi *frequent itemset*. Proses ini terus berulang secara iteratif dengan menaikkan tingkat kombinasi ($k = k + 1$) hingga tidak ada lagi kandidat baru yang terbentuk atau tidak ada kandidat yang memenuhi syarat batas minimum support.

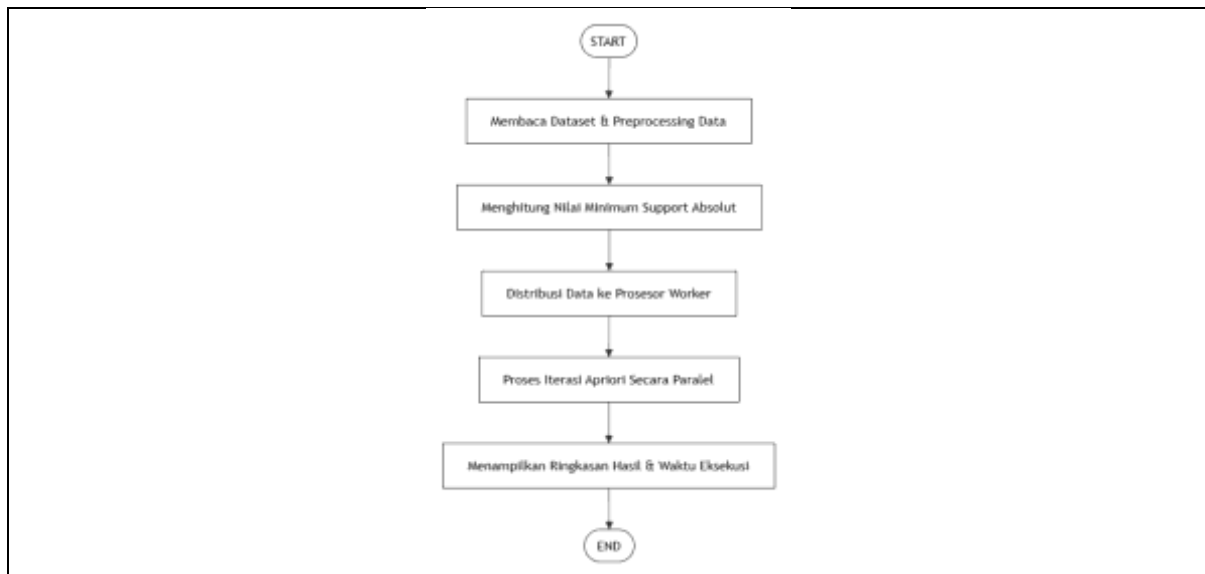
1. Desain Arsitektur Sistem Paralel

Sistem penambangan data transaksi ini dirancang secara terstruktur menggunakan arsitektur *Master-Worker* dengan memanfaatkan model eksekusi paralel SPMD (*Single Program, Multiple Data*) [3]. Dalam model arsitektur ini, peran masing-masing komponen prosesor dibagi menjadi dua fungsi utama:

- a. Master Node (Rank 0): Bertindak sebagai pimpinan pusat yang memegang kendali penuh terhadap manajemen data transaksi, interaksi masukan parameter operasional dari pengguna, pengombinasian kandidat global, serta finalisasi dan pelaporan hasil komputasi [4].
- b. Worker Node (Rank > 0): Bertindak sebagai unit pelaksana independen yang berfokus penuh untuk melakukan komputasi intensif dan pemindaian data secara mandiri pada partisi memori lokal yang dialokasikan kepada mereka masing-masing [3].

2. Alur Pelaksanaan Sistem

Tahapan pelaksanaan sistem penambangan data transaksi produk kebutuhan harian paralel dijalankan melalui alur komputasi sebagai berikut:



Gambar 1.1 Flowchart Alur Kerja Secara Umum

3. Langkah-Langkah Pelaksanaan (Algoritma Sistem):

a. Inisialisasi Sistem:

- i. Sistem menginisialisasi lingkungan kerja paralel MPI (*Message Passing Interface*) dan membaca total prosesor yang tersedia beserta nomor identifikasi (*rank*) masing-masing prosesor.
- ii. Seluruh prosesor melakukan sinkronisasi awal, kemudian mencatat waktu awal eksekusi program.

b. Persiapan Data (Khusus Prosesor Utama / Rank 0):

- i. Membaca file dataset transaksi sesuai batasan baris data yang dimasukkan melalui terminal.
- ii. Melakukan transformasi data (*preprocessing*) dengan mengelompokkan item belanjaan berdasarkan ID pelanggan menjadi format himpunan (*set*) transaksi.
- iii. Menghitung nilai ambang batas kemunculan absolut berdasarkan rumus:

$$\text{minimal_muncul} = \frac{\text{persen_minimum}}{100} \times \text{total_transaksi}$$

- iv. Memecah kumpulan data transaksi secara merata menjadi potongan-potongan kecil (*chunks*) sebanyak jumlah prosesor yang aktif.

c. Distribusi Data Awal:

- i. Prosesor Utama mengirimkan nilai ambang batas kemunculan absolut kepada seluruh prosesor *worker*.
- ii. Prosesor Utama membagikan (*scatter*) potongan data transaksi secara adil, sehingga setiap prosesor memegang sebagian data untuk diperiksa secara mandiri.

d. Perulangan Pencarian Pola Kombinasi (Dimulai dari level $k = 1$):

- i. Tahap A (Pembentukan Kandidat di Rank 0):

- 1) Jika $k = 1$, Prosesor Utama mendata seluruh jenis barang unik dari dataset sebagai kandidat awal (C_1).
 - 2) Jika $k > 1$, Prosesor Utama mengombinasikan daftar barang yang berhasil lolos dari level sebelumnya (L_{k-1}) menggunakan prinsip pemangkasan Apriori untuk membentuk kandidat baru (C_k).
- ii. Tahap B (Pengiriman Informasi):
- 1) Prosesor Utama mengirimkan daftar kandidat kombinasi barang (C_k) tersebut ke seluruh prosesor *worker*.
 - 2) Kondisi Berhenti 1: Jika daftar kandidat kosong atau tidak ada kombinasi baru yang bisa dibentuk, proses perulangan langsung dihentikan.
- iii. Tahap C (Perhitungan Frekuensi Lokal):
- 1) Setiap prosesor secara paralel menghitung seberapa sering kandidat kombinasi barang (C_k) muncul di dalam potongan data transaksi lokal mereka masing-masing menggunakan fungsi himpunan bagian (*subset*).
- iv. Tahap D (Reduksi & Penggabungan Global):
- 1) Seluruh hasil hitungan lokal dari masing-masing prosesor dikumpulkan kembali dan dijumlahkan secara otomatis ke Prosesor Utama.
- v. Tahap E (Penyaringan di Rank 0):
- 1) Prosesor Utama mengevaluasi hasil penjumlahan total frekuensi tersebut.
 - 2) Kandidat yang total frekuensinya lebih besar atau sama dengan nilai ambang batas kemunculan akan dimasukkan ke dalam daftar itemset lolos (L_k).
 - 3) Prosesor Utama membagikan daftar itemset yang lolos (L_k) ini ke seluruh prosesor agar sinkron.
 - 4) Kondisi Berhenti 2: Jika tidak ada satu pun kandidat yang berhasil lolos pada tahap penyaringan ini, proses perulangan dihentikan.
- vi. Tahap F (Iterasi Naik):
- 1) Nilai level dinaikkan ($k = k + 1$) untuk mencari kombinasi barang yang lebih kompleks, lalu sistem kembali melakukan langkah perulangan utama.
- e. Penyelesaian & Output:
- i. Seluruh prosesor melakukan sinkronisasi akhir untuk memastikan semua tahapan komputasi telah selesai.
 - ii. Prosesor Utama menghentikan pencatatan waktu dan menghitung selisih total waktu pemrosesan.
 - iii. Prosesor Utama menampilkan ringkasan tabel kombinasi produk yang paling sering dibeli beserta informasi efisiensi waktu eksekusi paralel program.

BAB IV

HASIL DAN PEMBAHASAN

1. Spesifikasi Lingkungan Pengujian Dan Dataset

Eksperimen komputasi paralel untuk algoritma Apriori ini dilakukan menggunakan satu perangkat komputer tunggal dengan spesifikasi teknis sebagai berikut:

- a. Perangkat Keras (Hardware): Laptop LENOVO (Model 81WA), Prosesor Intel(R) Core(TM) i3-10110U @ 2.10GHz (2 Cores Fisik / 4 Threads), RAM 8 GB, dan GPU NVIDIA GeForce MX.
- b. Perangkat Lunak (Software): Windows 11 Home, Python 3, dan pustaka mpi4py sebagai pengikat (*binding*) standar komunikasi MPI.

Dataset utama yang digunakan adalah Groceries_dataset.csv dari Kaggle yang berisi riwayat penjualan retail harian. Melalui augmentasi data menggunakan AI, dataset tersebut diperbesar menjadi lima variasi skala volume untuk menguji batas performa dan skalabilitas sistem, yaitu: 1.000.000, 1.500.000, 2.000.000, 2.500.000, dan 3.000.000 baris data. Seluruh skenario pengujian dijalankan konstan dengan parameter *Minimum Support* sebesar 3%.

2. Alur Fungsi Komunikasi Kolektif MPI

Implementasi pemrosesan paralel pada program apriori.py memanfaatkan fungsi kolektif dari pustaka mpi4py untuk mengatur pembagian kerja memori terdistribusi:

- a. MPI.COMM_WORLD: Bertindak sebagai komunikator utama untuk mengelola sinkronisasi global dari seluruh prosesor aktif yang dialokasikan via terminal (mpiexec).
- b. comm.Get_rank() dan comm.Get_size(): Digunakan untuk manajemen identitas. Proses dengan rank = 0 (Master Node) bertanggung jawab atas manajemen berkas I/O, kalkulasi kandidat (generate_candidates), membagi beban data, dan mencetak hasil akhir. Proses dengan rank > 0 (Worker Node) hanya fokus pada kalkulasi lokal.
- c. comm.scatter() dan comm.bcast(): Master Node memecah dataset menjadi potongan seimbang (*chunks*) dan mendistribusikannya ke seluruh *Worker* melalui comm.scatter(). Fungsi comm.bcast() digunakan untuk menyiarkan nilai *Minimum Support* global dan daftar kandidat barang secara seragam.
- d. comm.reduce() dan comm.Barrier(): Setelah *Worker* selesai menghitung frekuensi lokal menggunakan objek Counter(), fungsi comm.reduce() dengan operasi op=MPI.SUM dipanggil untuk mengumpulkan dan menjumlahkan seluruh hitungan parsial dari setiap *node* ke Master Node. Fungsi comm.Barrier() digunakan sebagai pembatas sinkronisasi tahap komputasi.
- e. MPI.Wtime() dan comm.Abort(): Waktu eksekusi diukur akurat lewat MPI.Wtime(). Fungsi comm.Abort() disematkan dalam blok try-except untuk menghentikan program secara paksa jika terjadi kegagalan pemuatan berkas guna menghindari error beruntun (*cascade error*).

3. Analisis Waktu Eksekusi

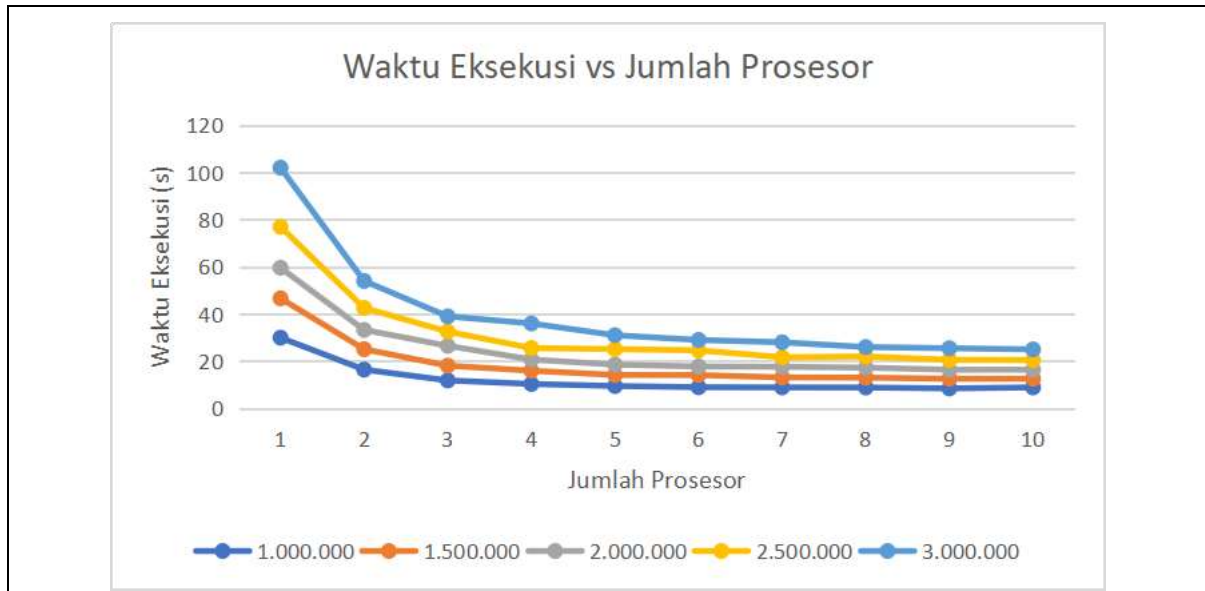
Pengujian dilakukan dengan memvariasikan alokasi jumlah prosesor dari 1 prosesor (serial murni) hingga maksimal 10 prosesor paralel. Rekam data riil waktu eksekusi (dalam satuan detik) disajikan pada Tabel 1.1.

Tabel 1.1 Data Riil Waktu Eksekusi Berdasarkan Jumlah Data dan Jumlah Prosesor (Detik)

Prosesor	1.000.000	1.500.000	2.000.000	2.500.000	3.000.000
1	29.96	46.6354	59.5888	76.9588	102
2	16.4217	25.081	33.2534	42.5501	54
3	11.8611	18.1194	26.5273	32.5452	39
4	10.359	15.977	20.7018	25.5816	36

5	9.4861	14.172	18.5439	25.1049	31
6	8.9859	14.1302	17.7686	24.5975	29
7	8.9023	13.1291	17.6351	21.6616	28
8	8.8606	13.0197	17.2631	21.9979	26
9	8.4745	12.5743	16.3832	20.5767	25.5
10	8.8968	12.5908	16.4179	20.5	25

Berdasarkan paparan data pada **Tabel 1.1**, pergerakan penurunan waktu eksekusi program secara visual dapat dilihat pada grafik di bawah ini:



Gambar 1.1 Grafik Perbandingan Waktu Eksekusi terhadap Jumlah Prosesor

Berdasarkan **Tabel 1.1** dan tampilan grafik pada **Gambar 1.1**, pemrosesan serial (1 prosesor) mencatatkan waktu eksekusi paling lambat di seluruh skala data, dengan puncaknya selama 102 detik pada volume 3.000.000 data. Hal ini dipicu oleh beban tunggal prosesor yang harus memindai jutaan baris data secara berulang di memori utama.

Ketika jumlah prosesor ditingkatkan, efisiensi waktu merosot sangat tajam. Penurunan paling signifikan terlihat pada transisi ke 2 prosesor, di mana pada skala data maksimal 3 juta baris, waktu langsung terpankas setengahnya dari 102 detik menjadi 54 detik. Tren performa optimal ini berjalan konsisten hingga alokasi 4 prosesor (36 detik pada data 3 juta). Efisiensi tinggi pada rentang 1 hingga 4 prosesor ini sangat bersesuaian dengan kapasitas fisik arsitektur Intel Core i3 laptop Lenovo yang digunakan, yang dibekali oleh teknologi *Hyper-Threading* berkekuatan 4 *logical processors (threads)*. Potongan data disebar merata via `comm.scatter()` ke tiap thread fisik sehingga pengerjaan berjalan simultan.

Sebaliknya, performa kecepatan mulai melandai dan cenderung stagnan pada penggunaan 5 hingga 10 prosesor. Titik komputasi tercepat mutlak memang berhasil diraih pada konfigurasi 10 prosesor dengan catatan waktu 25 detik (memotong lebih dari 75% waktu serialnya). Namun, penambahan prosesor di atas ambang batas *thread* fisik (> 4) memaksa sistem operasi melakukan *oversubscription* dan *context switching* yang intensif. Prosesor virtual MPI harus saling mengantre menggunakan core fisik yang terbatas, serta memunculkan *overhead* komunikasi dan latensi sinkronisasi (`comm.reduce()`) yang membuat kurva kecepatan cenderung stabil dan tidak menurun secara drastis lagi.

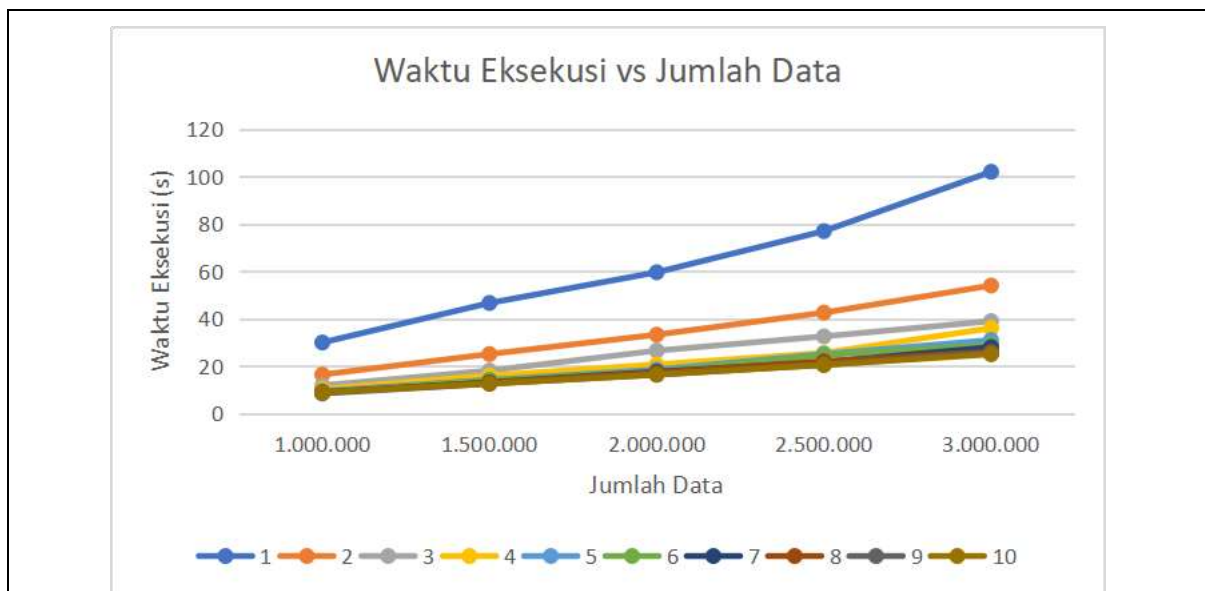
4. Analisis Speedup dan Efisiensi

Efektivitas performa sistem paralel ini diukur lewat dua metrik utama, yaitu *Speedup* (percepatan) dengan rumus $S_{(p)} = \frac{T_1}{T_p}$ dan *Efficiency* (efisiensi resource) dengan rumus $E_{(p)} = \frac{S_p}{p} \times 100\%$. Hasil kalkulasi metrik pada skala uji maksimal 3.000.000 data ($T_1 = 102$ detik) disajikan pada **Tabel 1.2**

Tabel 1.2 Hasil Kalkulasi Nilai Speedup dan Efficiency pada Skala 3.000.000 Data

Jumlah Prosesor (p)	Waktu Eksekusi (Detik)	Nilai Speedup S(p)	Persentase Efisiensi E(p)
1 (Serial)	102,00	1,00x	100,00%
2	54,00	1,89x	94,44%
3	39,00	2,62x	87,18%
4	36,00	2,83x	70,83%
5	31,00	3,29x	65,81%
6	29,00	3,52x	58,62%
7	28,00	3,64x	52,04%
8	26,00	3,92x	49,04%
9	25,50	4,00x	44,44%
10	25,00	4,08x	40,80%

Untuk melihat tren pertumbuhan percepatan sistem dan penurunan efisiensi secara lebih jelas, berikut disajikan grafik analisis performa paralel pada skala puncak 3.000.000 data:



Gambar 1.2. Grafik Skalabilitas Waktu Eksekusi terhadap Volume Data

Berdasarkan **Tabel 1.2** dan skema grafik dari **Gambar 1.2**, dapat diamati dengan jelas bahwa algoritma Apriori paralel berbasis MPI yang diimplementasikan memiliki tingkat skalabilitas yang sangat stabil dan linier. Saat volume dataset ditingkatkan secara konstan dengan penambahan masing-masing 500.000 baris data atau peningkatan beban kerja hingga 300%, grafik waktu eksekusi bergerak naik secara teratur membentuk garis diagonal linier, bukan melonjak secara eksponensial ekstrem yang berpotensi merusak atau membuat sistem *crash*. Sebagai contoh nyata, pada konfigurasi 4 prosesor, peningkatan waktu berjalan sangat konstan seiring bertambahnya data, yakni 10,35 detik pada 1 juta data, 15,97 detik pada 1,5 juta data, 20,70 detik pada 2 juta data, 25,58 detik pada 2,5 juta data, hingga 36 detik pada 3 juta data. Kestabilan pertumbuhan waktu yang linier pada **Gambar 1.2** membuktikan bahwa strategi partisi data yang diterapkan berhasil membagi beban kerja komputasi secara adil ke memori lokal. Pemrosesan paralel pada memori terdistribusi ini terbukti andal dalam menjaga

kestabilan penggunaan memori kerja pada laptop dengan kapasitas RAM 8 GB yang terbatas. Melalui pemrosesan paralel ini, risiko kegagalan fatal seperti sistem macet atau *Out-of-Memory* (OOM) yang biasa menghantui pemrosesan sekuensial algoritma Apriori konvensional saat dipaksa memproses data berskala jutaan baris dapat dimitigasi dan dihindari sepenuhnya.

5. Analisis Hasil Market Basket Analysis

Proses penggalian aturan asosiasi dengan ambang batas *minimum support* 3% berhasil menyaring kombinasi produk kebutuhan pokok harian yang paling sering dibeli bersamaan oleh konsumen. Pola *frequent itemset* yang dominan lolos dari tahapan pemangkasan (*pruning*) pada iterasi tingkat dua ($k = 2$) dirangkum pada **Tabel 1.3**.

Tabel 1.3 Sampel Kombinasi Produk Populer (Frequent Itemset) Lolos Batas Support 3%

No	Kombinasi Produk (Frequent Itemset)	Kategori	Implikasi Bisnis Retail
1	{Yogurt, Whole Milk}	<i>2-Itemset</i>	Menunjukkan kedekatan konsumsi produk olahan susu (<i>dairy product</i>). Pengelola retail dapat menata produk yogurt berdekatan dengan lemari pendingin susu murni.
2	{Other Vegetables, Whole Milk}	<i>2-Itemset</i>	Menunjukkan pola belanja harian komoditas utama. Penataan letak rak sayur segar dapat dibuat searah menuju area produk susu harian untuk mendorong <i>impulse buying</i> .
3	{Rolls/Buns, Whole Milk}	<i>2-Itemset</i>	Menunjukkan kombinasi menu pagi yang populer bagi konsumen. Strategi promosi paket diskon bundling dapat diterapkan pada kedua item ini.

Melalui hasil penambahan data di atas, produk *Whole Milk* (susu murni) konsisten muncul sebagai item pendukung utama yang memiliki keterikatan kuat dengan barang kebutuhan harian lainnya. Penemuan ini memberikan landasan analitik yang objektif bagi pihak manajemen ritel dalam mengoptimalkan tata letak barang (*product placement*) toko maupun menyusun program pemasaran komersial.

BAB V PENUTUP

1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian komputasi paralel pada algoritma Apriori menggunakan pustaka mpi4py yang telah dilakukan, dapat disimpulkan bahwa penerapan strategi *Count Distribution* terbukti sangat efektif dalam mereduksi waktu eksekusi pengolahan data transaksi retail berskala besar hingga mencapai 3.000.000 baris data.

Pengujian secara konsisten menunjukkan bahwa titik lompatan efisiensi yang paling signifikan secara performa perangkat keras terjadi pada konfigurasi 2 prosesor, di mana pemanfaatan *resource* CPU mencatatkan persentase efisiensi tertinggi sebesar 94,44% dan mampu memangkas waktu secara drastis sebesar 48 detik dengan capaian *Speedup* 1,89 kali lipat. Sementara itu, konfigurasi 4 prosesor bertindak sebagai batas efisiensi fisik perangkat keras yang paling seimbang (*equilibrium state*) dengan memanfaatkan seluruh 4 *logical threads* Intel Core i3 laptop LENOVO secara penuh, menghasilkan catatan waktu 36 detik dan tingkat efisiensi yang masih produktif sebesar 70,83%.

Meskipun catatan waktu tercepat secara mutlak diraih pada konfigurasi 10 prosesor dengan durasi 25 detik (*Speedup* maksimal 4,08x), namun secara teori komputasi paralel titik tersebut sudah tidak lagi efisien karena nilai efisiensinya mengalami degradasi hingga menyentuh angka 40,80%. Penurunan drastis ini menjadi bukti nyata dari berlakunya Hukum Amdahl, di mana pemaksaan jumlah proses MPI yang melebihi thread fisik memicu fenomena *oversubscription* dan antrean *context switching* yang intensif, serta membuat *overhead* komunikasi global melalui fungsi kolektif `comm.reduce()` menjadi jauh lebih dominan daripada beban komputasi datanya sendiri.

Terakhir, dari aspek *data mining*, sistem ini berhasil mengidentifikasi kombinasi produk harian (*frequent 2-itemset*) yang populer berdasarkan batas *Minimum Support* 3%, seperti kombinasi {Yogurt, Whole Milk}, {Other Vegetables, Whole Milk}, dan {Rolls/Buns, Whole Milk} yang memberikan wawasan valid bagi manajemen retail dalam mengoptimalkan strategi penataan tata letak barang (*product placement*) toko.

2. Saran

Berdasarkan hasil implementasi dan kesimpulan yang diperoleh dari proyek pemrosesan paralel ini, beberapa saran yang dapat diajukan untuk pengembangan penelitian atau sistem lebih lanjut di masa mendatang adalah sebagai berikut:

- a. Pengujian pada Lingkungan *Multi-Node Cluster*: Pengujian berikutnya disarankan tidak hanya dilakukan pada lingkungan simulasi *multi-core* lokal di satu perangkat laptop tunggal, melainkan dapat diimplementasikan pada kluster fisik komputer yang sesungguhnya (*multi-node cluster network*) untuk melihat performa skalabilitas paralel yang lebih ekstrem.
- b. Optimasi Algoritma Pembentukan Kandidat: Perlu dilakukan optimasi lanjutan pada fungsi pembangkitan kandidat (*candidate generation*), misalnya dengan mengombinasikan arsitektur paralel MPI ini dengan metode *Frequent Pattern Growth* (FP-Growth) paralel agar penggunaan memori global jauh lebih hemat saat menangani dataset transaksi yang melonjak hingga puluhan juta baris.
- c. Penerapan Mekanisme *Dynamic Load Balancing*: Mengingat ukuran potongan data transaksi lokal (*chunk data*) antarprosesor bisa menghasilkan beban komputasi yang bervariasi tergantung pada variasi item di dalamnya, penerapan strategi pembagian beban kerja secara dinamis (*dynamic load balancing*) sangat disarankan untuk menghindari adanya prosesor tertentu yang menganggur (*idle*) saat menunggu prosesor lain selesai pada fungsi `comm.Barrier()`.

- d. Pengembangan Antarmuka Pengguna (UI/Dashboard): Untuk implementasi praktis di bidang bisnis ritel, sistem analitik ini dapat diintegrasikan dengan antarmuka berbasis web atau *dashboard* interaktif, sehingga grafik performa komputasi dan tabel hasil *market basket analysis* dapat dibaca dengan mudah oleh pengguna awam tanpa harus melalui terminal *command prompt*.

LAMPIRAN

1. Kode Program

```
import pandas as pd
from mpi4py import MPI
from collections import Counter
import sys

# Inisialisasi MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

def print_ui(langkah, judul):
    if rank == 0:
        print(f"\n{'='*75}", flush=True)
        print(f"[LANGKAH {langkah}] {judul}", flush=True)
        print(f"{'='*75}", flush=True)
```

Kode program diawali dengan mengimpor beberapa pustaka utama yang dibutuhkan untuk pemrosesan data dan komputasi paralel. Pustaka *pandas* digunakan untuk memanipulasi data tabular dari file CSV, sedangkan *mpi4py.MPI* bertindak sebagai pustaka inti untuk mengaktifkan standar komunikasi paralel berbasis *Message Passing Interface* (MPI). Objek *collections.Counter* digunakan untuk menghitung frekuensi objek secara cepat menggunakan struktur data *dictionary*, dan pustaka *sys* dipakai untuk menangkap argumen baris perintah langsung saat program dijalankan melalui terminal. Setelah pustaka siap, program mengaktifkan komunikator global *MPI.COMM_WORLD* ke dalam variabel *comm* sebagai jalur utama pertukaran data antarnode. Melalui komunikator ini, fungsi *comm.Get_rank()* dipanggil untuk mendapatkan nomor identitas unik (*rank*) dari setiap prosesor yang berjalan, di mana *rank 0* akan bertindak sebagai *Master Node* dan *rank* lebih besar dari 0 bertindak sebagai *Worker Node*. Fungsi *comm.Get_size()* digunakan untuk menangkap total jumlah prosesor (*size*) yang dialokasikan oleh pengguna dalam klaster. Selanjutnya, didefinisikan sebuah fungsi pembantu bernama *print_ui* yang bertugas mencetak batas visual teks di terminal guna mempermudah pemantauan alur kerja. Fungsi ini dilengkapi dengan pengondisian *if rank == 0*: untuk memastikan bahwa instruksi pencetakan log ke layar hanya dieksekusi oleh prosesor utama agar output terminal tetap rapi dan tidak tumpang tindih oleh proses prosesor lain, dibantu parameter *flush=True* agar teks langsung dialirkan tanpa tertahan di memori *buffer*.

```
# Fungsi untuk menghasilkan kombinasi kandidat (Ck) berdasarkan itemset yang lolos sebelumnya (L_k-1)
def generate_candidates(L_prev, k):
    candidates = set()
    n = len(L_prev)
    for i in range(n):
        for j in range(i + 1, n):
            # Ambil itemset dan urutkan
            set1 = list(L_prev[i])
            set2 = list(L_prev[j])
            set1.sort()
            set2.sort()

            # Gabungkan jika k-2 elemen pertamanya sama (Prinsip Apriori)
            if set1[:k-2] == set2[:k-2]:
                new_itemset = tuple(sorted(set(set1) | set(set2)))
                # Pastikan panjangnya sesuai dengan level saat ini (k)
                if len(new_itemset) == k:
                    candidates.add(new_itemset)
    return list(candidates)
```

Fungsi `generate_candidates` dirancang untuk membentuk daftar kombinasi kandidat baru (C_k) pada tingkat iterasi tertentu berdasarkan daftar kombinasi produk yang berhasil lolos pada iterasi tingkat sebelumnya (L_{k-1}). Logika fungsi ini mengimplementasikan prinsip pemangkasan properti Apriori (*downward closure property*) untuk mengoptimalkan efisiensi komputasi. Proses pembentukan dilakukan dengan menggunakan dua lapis perulangan (*nested loop*) untuk membandingkan setiap pasangan itemset yang ada di dalam variabel `L_prev`. Di dalam perulangan tersebut, pasangan itemset diubah menjadi tipe data *list* dan diurutkan secara alfabetis menggunakan metode `.sort()`. Kedua itemset tersebut hanya akan digabungkan jika memiliki elemen yang sama dari indeks awal hingga indeks $k-2$. Jika kondisi tersebut terpenuhi, kedua itemset digabungkan menggunakan operasi himpunan gabungan (`()`) untuk menghasilkan kombinasi baru berbentuk tipe data *tuple* yang diurutkan. Sebelum dimasukkan ke dalam objek `candidates` yang bertipe *set* (untuk menghindari duplikasi), sistem melakukan validasi untuk memastikan bahwa panjang dari kombinasi baru tersebut benar-benar sesuai dengan level iterasi yang sedang aktif (k). Setelah seluruh kombinasi berhasil dievaluasi, struktur data *set* dikonversi kembali menjadi tipe data *list* sebelum dikembalikan sebagai output fungsi.

```
chunks_dataset = None
dataset_bagian_saya = None
minimal_muncul = 0
total_transaksi = 0
persen_minimum = 3 # Ubah persentase minimum support di sini

# Sinkronisasi semua prosesor sebelum mulai menghitung waktu
comm.Barrier()
if rank == 0:
    start_time = MPI.Wtime()

if rank == 0:
    print_ui("1", "MEMBACA DATASET & MENGUBAH KE FORMAT TRANSAKSI")
    nama_file = 'Groceries_dataset_massive.csv'

    # 1. Tangkap parameter jumlah baris dari argumen terminal (sys.argv)
    # Jika tidak ada argumen, biarkan None agar membaca semua
    jumlah_baris = int(sys.argv[1]) if len(sys.argv) > 1 else None

    try:
        # 2. Batasi data yang dibaca menggunakan parameter nrows
        if jumlah_baris:
            df = pd.read_csv(nama_file, nrows=jumlah_baris)
            print(f"-> Berhasil membaca {jumlah_baris} baris dari file '{nama_file}'.")
        else:
            df = pd.read_csv(nama_file)
            print(f"-> Berhasil membaca file '{nama_file}' secara utuh.")

    except FileNotFoundError:
        print(f"[Error] File {nama_file} tidak ditemukan di direktori!")
        comm.Abort()
        sys.exit()

    kolom_id = 'Member_number'
    kolom_item = 'itemDescription'

    # Mengubah setiap transaksi menjadi struktur 'set' agar pengecekan subset lebih cepat
    dataset_utuh = df.groupby(kolom_id)[kolom_item].apply(lambda x: set(x).values.tolist())
    item_unik = list(df[kolom_item].unique())

    total_transaksi = len(dataset_utuh)
    minimal_muncul = (persen_minimum / 100) * total_transaksi

    print(f"-> Ditemukan total {total_transaksi} transaksi unik.")
    print(f"-> Ditemukan {len(item_unik)} jenis barang unik di seluruh dataset.")
```

```

    print(f"-> Batas minimum kemunculan ({persen_minimum}%): {minimal_muncul:.2f}
    transaksi.")

    print_ui("2", f"DISTRIBUSI DATA KE {size} PROSESOR (SCATTER)")
    print("-> Memecah dataset transaksi agar dikerjakan bersama-sama...")

    # Pembagian dataset
    chunks_dataset = [[] for _ in range(size)]
    for i, transaksi in enumerate(dataset_utuh):
        chunks_dataset[i % size].append(transaksi)

```

Tahap awal pengerjaan program dimulai dengan deklarasi variabel global dan inisialisasi parameter nilai *Minimum Support* yang ditetapkan sebesar 3% melalui variabel `persen_minimum`. Fungsi pembatas kolektif `comm.Barrier()` dipanggil untuk menyinkronkan seluruh prosesor yang aktif agar memulai pencatatan waktu secara serentak, di mana prosesor *rank 0* akan mengunci waktu awal menggunakan `MPI.Wtime()`. Selanjutnya, prosesor *rank 0* sebagai *Master Node* mengambil alih tugas pembacaan berkas dataset bernama `Groceries_dataset_massive.csv`. Jumlah baris data yang dimuat dikontrol secara dinamis menangkap argumen terminal Windows melalui `sys.argv[1]`. Jika argumen diisi, pustaka `pandas` akan membatasi pemuatan baris lewat parameter `nrows=jumlah_baris`, namun jika kosong, dataset akan dibaca secara utuh. Blok pelindung `try-except` dipasang untuk menangani galat jika berkas data tidak ditemukan; apabila galat terjadi, sistem akan memanggil perintah `comm.Abort()` untuk menghentikan seluruh jalannya program di semua prosesor seketika guna menghindari *cascade error*. Setelah data tabular berhasil dimuat ke dalam *dataframe*, proses *preprocessing* dilakukan dengan mengelompokkan data berdasarkan indeks `Member_number` dan mengagregasi string produk pada kolom `itemDescription` ke dalam struktur data *set* melalui fungsi *lambda*. Hasil konversi ini disimpan ke dalam variabel `dataset_utuh` dalam bentuk *list* transaksi multi-dimensi. *Master Node* kemudian mengekstrak daftar barang unik, menghitung nilai total transaksi belanja yang valid, serta mengonversi persentase nilai acuan menjadi angka ambang batas kemunculan absolut global ke dalam variabel `minimal_muncul`. Langkah penutup pada blok ini adalah pembentukan partisi data seimbang (*data partitioning*), di mana berkas transaksi dipotong menjadi beberapa bagian (*chunks*) dengan metode pembagian modulus (`i % size`) yang disimpan ke dalam variabel array `chunks_dataset`.

```

minimal_muncul = comm.bcast(minimal_muncul, root=0)

# Scatter dataset transaksi ke masing-masing prosesor (hanya dilakukan sekali)
dataset_bagian_saya = comm.scatter(chunks_dataset, root=0)

t_awal = MPI.Wtime()
while MPI.Wtime() - t_awal < (rank * 0.1): pass
print(f"    [Prosesor {rank}] Menerima {len(dataset_bagian_saya)} transaksi untuk
    diperiksa.", flush=True)

comm.Barrier()

```

Setelah prosesor utama (*rank 0*) selesai menyiapkan parameter komputasi dan mempartisi dataset, langkah berikutnya adalah mendistribusikan data tersebut ke seluruh ekosistem komputasi paralel. Fungsi komunikasi kolektif `comm.bcast` dipanggil dengan parameter `root=0` untuk menyiarkan nilai ambang batas kemunculan absolut global (`minimal_muncul`) dari prosesor utama ke seluruh prosesor *worker* secara serentak agar acuan nilai penyaringan menjadi seragam. Setelah parameter tersinkronisasi, program memanggil fungsi kolektif `comm.scatter` untuk mendistribusikan potongan data transaksi belanja harian dari array `chunks_dataset` milik *rank 0* secara adil ke masing-masing memori lokal prosesor. Objek potongan data tersebut ditangkap oleh tiap-tiap prosesor ke dalam variabel lokal `dataset_bagian_saya`. Untuk merapikan visualisasi keluaran pada layar terminal, disematkan baris perulangan penahan waktu singkat (*delay*) berbasis fungsi `MPI.Wtime()` yang disesuaikan dengan urutan nomor *rank* prosesor. Mekanisme ini bertujuan memberikan jeda agar log pencetakan informasi jumlah data transaksi yang diterima oleh masing-masing prosesor dapat tampil berurutan di layar tanpa saling bertumpukan. Blok distribusi data ini ditutup dengan memanggil kembali

fungsi sinkronisasi `comm.Barrier()` untuk mengunci pergerakan langkah kerja agar seluruh prosesor berada pada kondisi siap sebelum memasuki perulangan penambangan data utama.

```
k = 1
L_prev = [] # Menyimpan itemset yang lolos pada tahap sebelumnya

while True:
    kandidat_utuh = []

    if rank == 0:
        print_ui(f"3.{k}", f"PEMBENTUKAN KANDIDAT ITEMSET (TAHAP C{k})")
        if k == 1:
            # Tahap 1: Kombinasi 1 barang
            kandidat_utuh = [(item,) for item in item_unik if pd.isna(item)]
        else:
            # Tahap > 1: Bangun kandidat berdasarkan itemset yang lolos (L_prev)
            kandidat_utuh = generate_candidates(L_prev, k)

        print(f"-> Total Kandidat (C{k}) yang akan dicek: {len(kandidat_utuh)} kombinasi.")

        # Broadcast kandidat Ck ke semua prosesor
        kandidat_utuh = comm.bcast(kandidat_utuh, root=0)

        # HENTIKAN LOOP JIKA TIDAK ADA KANDIDAT YANG BISA DIBENTUK
        if len(kandidat_utuh) == 0:
            if rank == 0:
                print(f"-> Tidak ada lagi kandidat yang bisa dibentuk untuk level {k}. Proses iterasi berhenti.")
            break
```

Tahap inti dari penambangan pola kombinasi produk dijalankan melalui perulangan tanpa batas (`while True:`) dengan inisialisasi awal tingkat kombinasi produk dimulai dari level $k = 1$. Variabel `L_prev` diinisialisasi sebagai array kosong untuk menyimpan daftar kombinasi barang yang berhasil lolos evaluasi pada iterasi sebelumnya. Di setiap awal perulangan, variabel global `kandidat_utuh` dikosongkan. Prosesor utama (*rank 0*) kemudian mengambil kendali untuk membangkitkan daftar kandidat kombinasi barang global (C_k). Jika kondisi iterasi berada pada tingkat pertama ($k = 1$), prosesor utama akan menyusun kandidat C_1 yang berisi daftar barang tunggal yang diekstrak dari seluruh item unik dataset dengan memfilter nilai kosong (*NaN*). Namun, jika iterasi telah berada pada tingkat lanjutan ($k > 1$), prosesor utama akan memanggil fungsi `generate_candidates` dengan menyertakan parameter `L_prev` dan level k yang aktif untuk mengombinasikan produk yang lolos sebelumnya. Daftar kandidat global yang berhasil disusun kemudian disebar secara merata dari *rank 0* ke seluruh prosesor kerja memanfaatkan fungsi komunikasi kolektif `comm.bcast`. Segera setelah proses siaran selesai, program melakukan pemeriksaan terhadap kuantitas panjang array kandidat menggunakan fungsi `len(kandidat_utuh)`. Jika jumlah kandidat yang terbentuk bernilai 0, sistem mengidentifikasi bahwa proses pencarian kombinasi telah mencapai batas maksimal, sehingga program akan mencetak log pemberitahuan di sisi *Master Node* dan memutus perulangan menggunakan perintah `break`.

```
# PERHITUNGAN LOKAL (Lakukan di masing-masing prosesor)
hasil_hitung_parsial = Counter()
for barang in kandidat_utuh:
    jumlah = 0
    set_barang = set(barang)
    for transaksi in dataset_bagian_saya:
        # Pengecekan subset (jauh lebih cepat menggunakan himpunan / set)
        if set_barang.issubset(transaksi):
            jumlah += 1
    if jumlah > 0:
        hasil_hitung_parsial[barang] = jumlah
```


Setelah menerima daftar kandidat kombinasi barang global (C_k) melalui operasi siaran, seluruh prosesor yang aktif di dalam sistem komputasi paralel akan menjalankan fungsi perhitungan frekuensi kemunculan secara mandiri dan simultan pada memori lokal mereka. Setiap prosesor menginisialisasi objek Counter lokal ke dalam variabel `hasil_hitung_parsial` untuk mencatat jumlah kemunculan barang. Proses kalkulasi dilakukan dengan melakukan iterasi pada setiap objek kombinasi produk yang ada di dalam variabel `kandidat_utuh`. Kombinasi produk berupa tipe data *tuple* tersebut dikonversi sementara menjadi tipe data *set* melalui variabel `set_barang`. Setiap prosesor kemudian melakukan pemindaian mendalam ke dalam potongan data transaksi lokal mereka (`dataset_bagian_saya`). Pencarian frekuensi dioptimalkan dengan memanfaatkan fungsi bawaan Python `.issubset()`, yang mengecek apakah kombinasi barang kandidat terkandung di dalam rekaman riwayat transaksi pelanggan. Jika fungsi subset mengembalikan nilai benar (*True*), nilai penghitung variabel jumlah akan bertambah 1. Setelah pemindaian potongan data selesai, jika nilai frekuensi akhir kombinasi barang tersebut lebih besar dari nol, hasil hitungan parsial tersebut akan disimpan ke dalam objek kamus `hasil_hitung_parsial` dengan kombinasi produk bertindak sebagai kunci (*key*) dan nilai akumulasi frekuensi bertindak sebagai nilai (*value*).

```
# PENGGABUNGAN HASIL (REDUCE KE RANK 0)
hasil_akhir = comm.reduce(hasil_hitung_parsial, op=MPI.SUM, root=0)

L_current = []
if rank == 0:
    print(f"\n[Tahap      L{k}]      Evaluasi      Minimum      Support      (>=
{minimal_muncul:.2f})")

    lolos = []
    tidak_lolos = []

    for barang in kandidat_utuh:
        jumlah = hasil_akhir.get(barang, 0)
        if jumlah >= minimal_muncul:
            L_current.append(barang)
            lolos.append(f"      [v] {barang} : {jumlah} kali")
        else:
            tidak_lolos.append(f"      [x] {barang} : {jumlah} kali")

    print(f"-> Ditemukan {len(L_current)} kombinasi yang LOLOS (Frequent {k}-
Itemsets):")
    # Tampilkan maksimal 10 agar terminal tidak penuh (khusus C1 biasanya
    sangat banyak)
    for teks in lolos[:10]:
        print(teks)
    if len(lolos) > 10:
        print(f"      ... (dan {len(lolos) - 10} kombinasi lolos lainnya)")

    print(f"-> Ditemukan {len(tidak_lolos)} kombinasi yang TIDAK LOLOS
(Pruned).")
```

Setelah seluruh prosesor menyelesaikan perhitungan frekuensi lokal, program memanggil fungsi komunikasi kolektif `comm.reduce` untuk mengonsolidasikan data. Melalui fungsi reduksi ini, objek kamus `hasil_hitung_parsial` dari seluruh *Worker Node* dikirimkan kembali ke prosesor utama (*rank 0*) dengan menerapkan parameter operasi `op=MPI.SUM`, yang secara otomatis menjumlahkan nilai kemunculan untuk setiap kunci kombinasi produk yang sama secara paralel. Hasil akumulasi frekuensi global tersebut diserahkan secara eksklusif kepada variabel `hasil_akhir` di sisi prosesor utama. Selanjutnya, *Master Node* menginisialisasi array kosong `L_current` serta dua array log visual bernama `lolos` dan `tidak_lolos`. Prosesor utama memindai ulang daftar kandidat global dan mengambil nilai total frekuensi kemunculannya menggunakan metode `.get(barang, 0)`. Nilai tersebut kemudian dievaluasi menggunakan operator kondisi terhadap ambang batas variabel `minimal_muncul`. Jika total kemunculan produk global memenuhi atau melampaui ambang batas absolut, kombinasi tersebut dinyatakan sah sebagai anggota pola populer (*Frequent k-Itemset*) dan dimasukkan ke dalam list `L_current`. Log

ringkasan hasil penyaringan dipisahkan secara rapi ke layar terminal, di mana sistem membatasi tampilan visual maksimal hanya 10 baris kombinasi teratas menggunakan pemotongan indeks lolos[:10] untuk mencegah terjadinya penumpukan teks yang terlalu panjang pada layar terminal saat mengevaluasi iterasi level pertama.

```
# Broadcast L_current ke seluruh prosesor untuk di-loop di tahap k selanjutnya
L_current = comm.bcast(L_current, root=0)

# Update L_prev dengan kandidat yang lolos di tahap ini
L_prev = L_current

# HENTIKAN LOOP JIKA TIDAK ADA YANG LOLOS
if len(L_prev) == 0:
    if rank == 0:
        print(f"-> Tidak ada frequent itemset yang lolos di level {k}. Proses
iterasi berhenti.")
        break

    k += 1 # Lanjut ke level frekuensi selanjutnya (k+1)
    comm.Barrier()

comm.Barrier()
if rank == 0:
    end_time = MPI.Wtime()
    waktu_eksekusi = end_time - start_time

    print_ui("SELESAI", "PROSES APRIORI PARALEL BERAKHIR")
    print(f"[*] Frekuensi maksimal yang dicapai : {k - 1}-itemset")
    print(f"[*] Waktu Pemrosesan Total : {waktu_eksekusi:.4f} detik")
    print(f"{'='*75}\n")
```

Tahap akhir dari siklus perulangan ditujukan untuk menyinkronkan data *frequent itemset* yang berhasil lolos seleksi. Daftar kombinasi produk terpopuler yang tersimpan di dalam variabel `L_current` pada prosesor utama disiarkan kembali ke seluruh prosesor *worker* via fungsi kolektif `comm.bcast`. Nilai dari `L_current` kemudian disalin ke dalam variabel `L_prev` untuk digunakan sebagai fondasi pembuatan kombinasi kandidat baru pada iterasi tingkat berikutnya. Program kemudian melakukan pengecekan kondisi; jika panjang dari variabel `L_prev` bernilai 0 (artinya tidak ada satu pun kombinasi produk yang berhasil melampaui batas minimum support pada level tersebut), perulangan utama program akan dihentikan paksa melalui statement `break`. Namun, jika kombinasi produk yang lolos masih ditemukan, nilai tingkatan level akan dinaikkan (`k += 1`) dan fungsi pembatas `comm.Barrier()` dipanggil untuk menjaga sinkronisasi langkah eksekusi antarprosesor sebelum masuk ke putaran perulangan selanjutnya. Ketika perulangan telah berakhir sepenuhnya, fungsi `comm.Barrier()` dipanggil untuk terakhir kali sebagai pintu gerbang penutup komputasi paralel. Prosesor utama (*rank 0*) kemudian mencatat waktu akhir menggunakan fungsi `MPI.Wtime()`, menghitung selisih total durasi runtime komputasi paralel program ke dalam variabel `waktu_eksekusi`, serta menampilkan log ringkasan hasil akhir performa sistem yang mencakup tingkatan level kombinasi tertinggi yang berhasil dicapai beserta visualisasi total durasi pemrosesan dalam satuan detik.

DAFTAR PUSTAKA

- [1] Alfiyan, A. R., Kahfi, A. H., Kusumayudha, M. R., & Rezki, M. (2019). Analisis Market Basket Dengan Algoritma Apriori Pada Transaksi Penjualan Di Freshfood.
- [2] Azzeh, M., Nassif, A. B., & Banitaan, S. (2018). Comparative analysis of soft computing techniques for predicting software effort based on use case points. *IET Software*, 12(1), 19-29.
- [3] Fang, W., Lu, M., Xiao, X., He, B., & Luo, Q. (2019). Frequent itemset mining on graphics processors. *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN)*, 34-42.
- [4] Jiang, F., Leung, C. K., & Pazdor, A. G. M. (2017). Big data mining of large-scale datasets with a parallel and distributed approach. *Proceedings of the International Conference on Data Science and Advanced Analytics (DSAA)*, 532-541.
- [5] Kaur, M., & Kang, S. (2016). Market basket analysis: Identify the changing trends of market data using association rule mining. *Procedia Computer Science*, 85, 78-85.